macOS Endpoint Management Baseline in Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — macOS Endpoint Management Baseline (Microsoft Intune) 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Outcome . . . 6

1. Solution Overview — macOS Endpoint Management Baseline (Microsoft Intune) 6

What this solution is . . . 6

The macOS management model in Intune . . . 6

The two ways to configure macOS: settings catalog vs custom `.mobileconfig` . . . 6

Device restrictions . . . 7

Compliance policies . . . 7

macOS software-update management . . . 7

The compliance and drift model . . . 7

The deployment model . . . 8

Benefits . . . 8

Limitations (be honest with buyers) . . . 8

2. Requirements Guide — macOS Endpoint Management Baseline (Microsoft Intune) 8

2.1 Intune licensing . . . 9

2.2 Microsoft Entra ID with users/devices in scope . . . 9

2.3 Apple MDM Push (APNs) certificate . . . 9

2.4 macOS enrollment configured . . . 9

2.5 Supported macOS version . . . 10

2.6 Entra pilot and production device groups . . . 10

2.7 Baseline decisions agreed (Assess inputs) . . . 10

2.8 (Optional) Custom `.mobileconfig` payloads for non-catalog settings . . . 10

2.9 Change control / approval process . . . 11

Checklist . . . 11

Global Requirements — Intune Solutions (category-level) 11

G1. Microsoft Entra ID tenant . . . 11

G2. Microsoft Intune subscription . . . 11

G3. Administrative roles . . . 12

G4. Device enrollment in Intune . . . 12

G5. Pilot security group . . . 12

G6. Supported Windows baseline . . . 12

G7. Network access to Microsoft cloud endpoints . . . 12

3. Implementation Guide — macOS Endpoint Management Baseline (Microsoft Intune) 12
Step 1 — Stage the baseline decisions (Assess) . . . 13
Step 2 — Confirm Apple management prerequisites . . . 13
Step 3 — Create the device groups . . . 13
Step 4 — Baseline the tenant before you build (recommended) . . . 13
Step 5 — Build the baseline configuration profile (settings catalog) . . . 13
Step 6 — Build the restrictions policy (settings catalog) . . . 14
Step 7 — Build the macOS software-update policy (settings catalog) . . . 14
Step 8 — Create the compliance policy . . . 15
Step 9 — Add a custom `.mobileconfig` profile (only if needed) . . . 15
Step 10 — Apply and validate on the pilot, with pilot gate and rollback note . . . 15
Step 11 — Expand to production . . . 16
5. Testing Guide — macOS Endpoint Management Baseline (Microsoft Intune) 16
Test 1 — Baseline configuration profile applied to the pilot . . . 17
Test 2 — Restrictions policy applied . . . 17
Test 3 — Software-update policy applied . . . 17
Test 4 — Settings present on the device . . . 17
Test 5 — Compliance policy evaluates the device . . . 17
Test 6 — Custom `.mobileconfig` profile applied (if used) . . . 17
Test 7 — Workflow soak (the baseline did not break the business) . . . 17
Test 8 — Fleet posture and drift report . . . 17
Test 9 — Remediation dry-run (safe by default) . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — macOS Endpoint Management Baseline (Microsoft Intune) 18
A. Nothing applies to any Mac . . . 18
B. A single Mac is not applying the baseline . . . 19
C. Device status shows "Error" or "Conflict" . . . 19
D. Restrictions not taking effect . . . 19
E. Software updates not enforcing . . . 19
F. Compliance policy marks Macs non-compliant unexpectedly . . . 20
G. Custom `.mobileconfig` profile shows applied but the setting is wrong . . . 20
H. Drift after a Mac was compliant . . . 20
I. Scripts fail to connect or return nothing . . . 20
J. Rollback . . . 21
7. Best Practices — macOS Endpoint Management Baseline (Microsoft Intune) 21
Naming and structure . . . 21

Prefer the settings catalog . . . 22
Avoid conflicting authorities . . . 22
Group and assignment strategy . . . 22
Pilot discipline . . . 22
Software updates . . . 23
Compliance and Conditional Access . . . 23
Ongoing operation . . . 23
Scope boundaries (don't reinvent other solutions) . . . 23
8. Deliverables Summary — macOS Endpoint Management Baseline (Microsoft Intune) 23
Documentation . . . 23
PowerShell scripts . . . 24
Object naming convention (used across docs and scripts) . . . 25
What the buyer receives . . . 25
Outcome . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — macOS Endpoint Management Baseline (Microsoft Intune)

Macs enrolled in Microsoft Intune **without a deliberate baseline drift** into inconsistent, non-compliant states — some have a screen-saver password, some don't; some auto-update, some sit years behind; ad-hoc profiles pile up and nobody can say which Macs meet the corporate standard. This package establishes the **foundational macOS management baseline** in Intune — configuration profiles (settings catalog and, where needed, custom `.mobileconfig`), device restrictions, a compliance policy, and a macOS software-update policy — so every Mac meets corporate standards for security, performance, and user experience, deployed pilot-first and rollback-ready.

This is a complete, **deploy-it-today** package written for an administrator with **no prior macOS management experience**. It is the *general* baseline — deliberately **not** enrollment/ADE setup and **not** a formal hardening benchmark (CIS/STIG); those are separate solutions this guide references rather than duplicates.

What you get

• A plain-language **Solution Overview** of how Intune manages macOS (MDM protocol, the Apple MDM Push

certificate, configuration profiles), the crucial choice between the **settings catalog** and a **custom** `.mobileconfig`, device restrictions, compliance policies, and macOS software-update management.

• A **Requirements Guide** with why / verify / implement-if-missing for every prerequisite — including

the Apple MDM Push (APNs) certificate, macOS enrollment, and the pilot/production groups — plus a consistent object-naming convention.

• An **Implementation Guide** of precise, click-by-click Intune steps to build the named baseline

configuration profile, restrictions policy, software-update policy, and compliance policy; assign to a pilot; validate; and expand — with a pilot gate and a rollback note.

• Five **PowerShell scripts** for baseline readiness discovery, per-device applied-state validation,

fleet posture and drift reporting, safe drift remediation, and repeatable group targeting.

• **Testing** and **Troubleshooting** guides for confident rollout and ongoing operation.

What this delivers

A **consistent, compliant Mac fleet** built as Intune settings-catalog configuration profiles (with a custom `.mobileconfig` only where the catalog cannot reach a payload), a device-restrictions policy, a macOS software-update policy, and a compliance policy — with provable per-device state, drift detection and remediation, and a pilot-first rollout.

The deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

Every Mac on a consistent, compliant Intune baseline — secure, performant, and predictable — proven on a pilot before you scale, and deployable by a single administrator.

1. Solution Overview — macOS Endpoint Management Baseline (Microsoft Intune)

What this solution is

This is the **foundational macOS management baseline** for Microsoft Intune: the deliberate set of configuration profiles, device restrictions, a compliance policy, and a macOS software-update policy that every corporate Mac should receive the moment it is under management. Macs enrolled in Intune without a baseline drift into inconsistent, non-compliant states — some have a screen-saver password, some don't; some auto-update, some sit years behind; nobody can say which Macs meet the corporate standard. This solution establishes that standard as named Intune policy, deploys it pilot-first, and proves it, so every Mac lands on a consistent, compliant, predictable configuration.

This is the **general baseline**. It is intentionally *not* an enrollment/Automated Device Enrollment (ADE) build, and it is *not* the CIS hardening benchmark — those are separate solutions. This one assumes your Macs are already enrolled and gives them a sensible, corporate-standard configuration. Where enrollment or a formal benchmark is relevant, this guide references it rather than duplicating it.

The macOS management model in Intune

macOS is not Windows. There is no ADMX, no Windows settings catalog, no Group Policy. Apple manages a Mac through the **MDM protocol**, and Intune configures it by delivering Apple **configuration profiles** — declarative payloads the Mac's management framework applies. Intune reaches a Mac only after two things exist: an **Apple MDM Push (APNs) certificate** (the trust channel between your tenant and Apple) and an **enrollment** that has brought the Mac under management. Once those are in place, everything in this baseline is delivered as Intune policy over that channel.

The two ways to configure macOS: settings catalog vs custom `.mobileconfig`

Every setting on this baseline is delivered as an Apple configuration-profile payload. There are two ways to author those payloads in Intune, and knowing which to use is central to this solution:

1. **Settings catalog (Apple)** — the primary, preferred surface. In

Devices → **macOS** → **Configuration** → **Create** → **New Policy** → **Settings catalog**, Microsoft exposes Apple's payloads in a searchable picker grouped by area (Login Window, Restrictions, Security, Software Update, Privacy Preferences, and more). You add settings and set their values in the GUI; Intune builds the `.mobileconfig` for you. Use the settings catalog for everything it can express — it is discoverable, versioned, and reports per-setting status.

1. **Custom configuration profile (`.mobileconfig`)** — the escape hatch. A minority of Apple payloads

are not (yet) surfaced in the settings catalog. For those, you author or obtain the raw `.mobileconfig` file and upload it via **Devices** → **macOS** → **Configuration** → **Create** → **New Policy** → **Templates** → **Custom**. This delivers the exact Apple payload the catalog cannot reach. Use it only when the catalog cannot express the setting, because a custom profile is opaque (Intune reports "applied", not per-setting detail).

A third surface — **macOS shell scripts** (Devices → macOS → Shell scripts) — exists for the rare control that is only enforceable by a command. The general baseline in this solution is built entirely from configuration profiles and does not require shell scripts; they are noted only so you know the surface exists.

Device restrictions

Restrictions are a specific, high-value slice of configuration-profile settings that turn features on or off at the OS level — for example controlling AirDrop, content caching, iCloud document sync, Game Center, or requiring a password after the screen saver begins. In the modern Intune console, macOS restrictions are authored **inside the settings catalog** (the older "Device restrictions" template is being consolidated into the catalog), so this solution builds them as a dedicated settings-catalog policy you can reason about and assign independently of the broader baseline.

Compliance policies

A **compliance policy** does not configure the Mac — it *evaluates* it and reports a compliant / non-compliant verdict. A macOS compliance policy checks conditions such as minimum OS version, whether **FileVault** disk encryption is on, whether the **firewall** is on, and system integrity. Compliance is what makes the baseline provable and what other systems (notably Conditional Access, a separate solution) consume to allow or block access. This baseline includes a macOS compliance policy so every Mac's posture is measured and reported, not just configured.

macOS software-update management

Keeping Macs current is part of the baseline, not an afterthought. Intune manages macOS updates through the **Software Update** payload (in the settings catalog) and, on modern macOS, through **Declarative Device Management (DDM)** update enforcement, where supported. You define whether the Mac downloads and installs OS and security updates automatically, and you can require a **minimum OS version** and set a **deadline** so updates are not indefinitely deferred. This solution builds a named macOS software-update policy so update behaviour is deliberate and consistent instead of left to each user.

> ***Version note:*** *the exact Software Update payload keys and the availability of DDM update enforcement depend on the macOS version and Apple's current MDM capabilities. Configure from the options Intune presents for your macOS releases; where a capability is version-dependent this guide flags it rather than assuming it.*

The compliance and drift model

Configuring the baseline is only half the job. You also:

- **Prove application** — Intune reports per-profile and (for settings-catalog policies) per-setting state, so you can show each Mac received the baseline.

- **Prove compliance** — the compliance policy gives each Mac a compliant / non-compliant verdict.
- **Detect drift** — a Mac can fall out of the intended state (a conflicting profile, a removed assignment, a failed apply, a user change). The included reporting and validation scripts surface drift so you can reapply.

The deployment model

> *Build the named baseline configuration profiles, the restrictions policy, the compliance policy, and the macOS software-update policy as Intune settings-catalog policies (with a custom `.mobileconfig` only where the catalog cannot reach a payload), assign to a pilot, prove application and compliance, then expand fleet-wide. The repeatable way:* ***Assess → Configure → Pilot → Validate → Expand.***

Benefits

- **Consistency** — every Mac lands on the same corporate-standard configuration instead of drifting.
- **Provable compliance** — per-device configuration state plus a compliant / non-compliant verdict.
- **Current by design** — a deliberate macOS software-update policy instead of per-user chaos.
- **Discoverable and maintainable** — built primarily in the settings catalog, named consistently, and assigned by group.
- **Safe rollout** — piloted before the fleet, with a rollback note, because even a baseline change can surprise a workflow.

Limitations (be honest with buyers)

- This is the **general baseline**, not enrollment/ADE setup and not a formal hardening benchmark (CIS/STIG) — those are separate solutions referenced here, not duplicated.
- Some Apple payloads are not in the settings catalog and require a custom `.mobileconfig`; the exact payload keys and values are deferred to Apple's public MDM documentation.
- Software-update enforcement capabilities (especially DDM-based) are **macOS-version-dependent**; configure from what Intune offers for your releases.
- A baseline reduces drift but does not replace ongoing operations — you still detect and remediate drift over time (the included scripts help).

2. Requirements Guide — macOS Endpoint Management Baseline (Microsoft Intune)

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Object names used throughout this solution (reuse them **verbatim** in Intune):

- Baseline settings-catalog policy: `macOS Baseline – Config – Pilot` (later `macOS Baseline – Config – Production`)
- Restrictions settings-catalog policy: `macOS Baseline – Restrictions – Pilot` / `macOS Baseline – Restrictions – Production`
- Software-update settings-catalog policy: `macOS Baseline – Software Update – Pilot` / `macOS Baseline – Software Update – Production`
- Custom `.mobileconfig` profile (only if needed): `macOS Baseline – Custom – Pilot` / `macOS Baseline – Custom – Production`
- Compliance policy: `macOS Baseline – Compliance`
- Device groups: `macOS Baseline – Pilot Devices` and `macOS Baseline – Production Devices`
- Version/scope tag (put in every policy/profile description): `macOS-Baseline-v1.0`

2.1 Intune licensing

Description: An Intune-bearing license (Plan 1 is sufficient for configuration profiles, custom profiles, compliance, and software-update policy). **Why required:** Intune delivers every configuration profile, the compliance policy, and the software-update policy in this baseline. **How to verify:** Microsoft 365 admin center → **Billing** → **Licenses**; confirm an Intune-bearing plan is present and assigned. **How to implement if missing:** Assign Microsoft 365 E3/E5 or Business Premium (Intune included), or standalone Intune Plan 1.

2.2 Microsoft Entra ID with users/devices in scope

Description: An Entra ID tenant whose Mac users and devices are the target of this baseline. **Why required:** Intune assigns policy to Entra groups, and enrolled Macs are represented as Entra devices. **How to verify:** Sign in to `https://entra.microsoft.com`; confirm the Mac users/devices exist. **How to implement if missing:** Ensure the Mac users are licensed Entra users and the Macs enroll into the same tenant (see 2.4).

2.3 Apple MDM Push (APNs) certificate

Description: A valid Apple Push Notification service (APNs) certificate configured in Intune. **Why required:** No Apple device — including a Mac — can be managed by Intune without an APNs certificate. Until it exists and is unexpired, **no** profile, compliance policy, or update policy reaches any Mac. **How to verify:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm it is present and **not expired**. Run `MACBASE-Discovery.ps1` for a readiness signal. **How to implement if missing:** Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate** → **download the CSR**, create the certificate at `https://identity.apple.com` with a long-lived, business-owned Apple ID, upload it back, and **record the renewal date** (the certificate expires annually; letting it lapse stops all Mac management).

2.4 macOS enrollment configured

Description: A working macOS enrollment method so target Macs are enrolled and manageable — Automated Device Enrollment (ADE/DEP), account-driven device enrollment, or Company Portal. **Why required:** The baseline profiles, compliance policy, and update policy apply only to enrolled Macs. This solution assumes enrollment is already in place (enrollment/ADE is a separate solution). **How to verify:** Intune → **Devices** → **macOS** → **macOS devices**; the target Macs appear as **managed**. Run `MACBASE-Discovery.ps1` for a fleet count and version spread. **How to implement if missing:** Configure **Automated Device Enrollment** (Apple Business/School Manager token + an enrollment profile) for corporate Macs, or account-driven / Company Portal enrollment for user-owned Macs, then enroll the devices. (Enrollment setup is out of scope for this baseline and covered by a separate solution.)

2.5 Supported macOS version

Description: A currently supported macOS major version on the target Macs. **Why required:** Some baseline payloads — especially software-update enforcement and Declarative Device Management update behaviour — depend on the macOS major version. Configuring against an unsupported version mismatches the available settings. **How to verify:** On a Mac, **Apple menu** → **About This Mac**; or Intune → device → **Hardware**. Run `MACBASE-Discovery.ps1` for the fleet version spread. **How to implement if missing:** Update the Macs to a currently supported macOS version; then build the software-update policy (Implementation Step 7) to keep them current.

2.6 Entra pilot and production device groups

Description: Two Entra ID security groups: `macOS Baseline – Pilot Devices` (a small, representative sample of Macs) and `macOS Baseline – Production Devices` (the remaining Mac fleet). **Why required:** The baseline is piloted before fleet rollout so a surprising setting is caught on a few Macs, not all of them. **How to verify:** Entra → **Groups**; both groups exist. **How to implement if missing:** Entra → **Groups** → **New group** → **Security**; add a few representative Macs to the pilot group and the rest to the production group. (Dynamic device groups filtered on `deviceOSType -eq "MacMDM"` are a convenient way to keep the production group current — verify the filter matches your tenant's device attributes.)

2.7 Baseline decisions agreed (Assess inputs)

Description: The specific baseline values you intend to enforce: which restrictions to apply (for example AirDrop, content caching, screen-saver password), the update behaviour (auto-download/install, minimum OS version, deadline), and the compliance thresholds (minimum OS version, FileVault required, firewall required). **Why required:** The baseline is a set of *decisions*, not defaults. Agreeing them up front (with the app/security owners) prevents a policy that blocks a legitimate workflow. **How to verify:** You can state, for each area, the value you will set and why. **How to implement if missing:** Hold a short Assess review with the security and app owners; record the agreed values (they become the settings you enter in Implementation Steps 5–8).

2.8 (Optional) Custom `.mobileconfig` payloads for non-catalog settings

Description: The `.mobileconfig` file for any baseline setting whose Apple payload is not present in the Intune settings catalog. **Why required:** A minority of Apple payloads can only be delivered via a custom configuration profile carrying the raw payload. **How to verify:** While building the settings-catalog policies, note any agreed setting with no catalog equivalent; those need a custom `.mobileconfig`. **How to**

implement if missing: Author the payload from Apple's public MDM documentation (or obtain a vetted `.mobileconfig`) and upload it via Intune → **Devices** → **macOS** → **Configuration** → **Create** → **Templates** → **Custom** (Implementation Step 9).

2.9 Change control / approval process

Description: A lightweight, documented approval step for applying and later changing the baseline. **Why required:** Baseline changes are production changes; they need review and a rollback plan. **How to verify:** You have a place to record approved changes and the baseline version in use (a change ticket, register, or the policy descriptions). **How to implement if missing:** Adopt a simple change record (who approved, what changed, rollback step). Record the tag `macOS-Baseline-v1.0` in each policy/profile description.

Checklist

- Intune licensing assigned
- Entra tenant with the Mac users/devices in scope
- Apple MDM Push (APNs) certificate present and **not expired** — `MACBASE-Discovery.ps1`
- macOS enrollment configured (ADE / account-driven / Company Portal); Macs enrolled — `MACBASE-Discovery.ps1`
- Supported macOS version on the target Macs
- `macOS Baseline – Pilot Devices` and `macOS Baseline – Production Devices` groups created
- Baseline decisions agreed (restrictions, update behaviour, compliance thresholds)
- (Optional) Custom `.mobileconfig` payloads identified for any non-catalog settings
- Change-control / approval process and version tag convention agreed

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune

Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — macOS Endpoint Management Baseline (Microsoft Intune)

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`). This builds the named baseline — configuration profile, restrictions policy, compliance policy, and macOS software-update policy — primarily in the **settings catalog** (with a custom `.mobileconfig` only where the catalog cannot reach a payload), proves it on a pilot, then expands fleet-wide.

> ***Model:*** *Assess → Configure → Pilot → Validate → Expand. A small* `macOS Baseline – Pilot Devices` *group validates the baseline before the* `macOS Baseline – Production Devices` *fleet*

*receives it. **Do not skip the pilot gate or the rollback note (Step 10).***

***Naming:** Create every object with the **exact** name shown and reuse it verbatim in the Testing and Troubleshooting guides and in the scripts. Put the tag `macOS-Baseline-v1.0` in every description.*

Step 1 — Stage the baseline decisions (Assess)

1. Confirm the agreed baseline values from Requirement 2.7: the **restrictions** to apply, the

software-update behaviour (auto-download/install, minimum OS version, deadline), and the **compliance** thresholds (minimum OS version, FileVault required, firewall required).

1. Sort each agreed setting into a surface: **settings catalog** (nearly all — Steps 5–8), or

custom `.mobileconfig` (only settings with no catalog payload — Step 9).

1. Record `macOS-Baseline-v1.0` as the tag you will place in each policy description.

Step 2 — Confirm Apple management prerequisites

1. Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm present and **not

expired** (Requirement 2.3).

1. Confirm the target Macs are enrolled: Intune → **Devices** → **macOS** → **macOS devices** (Requirement

2.4). Optionally run `MACBASE-Discovery.ps1` for a readiness snapshot.

Step 3 — Create the device groups

1. Entra (`https://entra.microsoft.com`) → **Groups** → **New group** → **Group type: Security** → name

`macOS Baseline – Pilot Devices` → add a few representative Macs. **Create**.

1. Create `macOS Baseline – Production Devices` for the rest of the fleet (a dynamic device group is

convenient here — see Requirement 2.6). **Create**.

Step 4 — Baseline the tenant before you build (recommended)

1. Run `MACBASE-Discovery.ps1` to list any **existing** macOS configuration policies, custom profiles,

compliance policies, and update policies. Note anything already targeting the same area so you do not create a **conflicting** second authority for the same setting (a leading cause of "Error / Conflict" status later).

Step 5 — Build the baseline configuration profile (settings catalog)

1. Intune → **Devices** → **macOS** → **Configuration** → **Create** → **New Policy**.
2. **Platform:** macOS. **Profile type: Settings catalog**. **Create**.

3. **Name:** `macOS Baseline – Config – Pilot`. **Description:** include the tag `macOS-Baseline-v1.0`.

Next.

1. Click **Add settings**. In the settings picker, search for and add the core baseline payloads, then set each to the agreed value. Typical baseline areas (configure the actual values you agreed in Step 1):

• **Login Window** — e.g. disable automatic login; show name/password fields (not a user list).

• **Security** — e.g. require a password after sleep or screen saver begins, and set the grace period; enable FileVault prompting (if you enforce encryption here rather than in a dedicated encryption profile).

• **Screen Saver / Password** — idle time before the screen saver, and password required to wake.

• **Privacy Preferences (PPPC)** — pre-approve management/security agents where needed (author from Apple's public payload documentation if you have such agents).

1. **Next** → **Scope tags** (optional) → **Next** → **Assignments** → **Add groups** → `macOS Baseline – Pilot Devices`. **Next** → **Review + create** → **Create**.

Step 6 — Build the restrictions policy (settings catalog)

1. Intune → **Devices → macOS → Configuration → Create → New Policy → Platform: macOS → Profile type: Settings catalog → Create**.

1. **Name:** `macOS Baseline – Restrictions – Pilot`. **Description:** tag `macOS-Baseline-v1.0`.

Next.

1. **Add settings** → search the **Restrictions** category and add the agreed toggles, for example:

• AirDrop (allow/deny), content caching (allow/deny), iCloud document/desktop sync, Game Center, media sharing, and similar feature restrictions per your agreed baseline.

1. **Assignments** → **Add groups** → `macOS Baseline – Pilot Devices`. **Review + create** → **Create**.

*Keeping restrictions in their **own** policy (separate from Step 5) lets you assign, adjust, or roll back restrictions independently of the broader configuration — recommended.*

Step 7 — Build the macOS software-update policy (settings catalog)

1. Intune → **Devices → macOS → Configuration → Create → New Policy → Platform: macOS → Profile type: Settings catalog → Create**.

1. **Name:** `macOS Baseline – Software Update – Pilot`. **Description:** tag `macOS-Baseline-v1.0`.

Next.

1. **Add settings** → search **Software Update** and add the agreed update controls, for example:

• Automatically **download** and **install** macOS and app/security updates.

• Enforce a **minimum OS version** and (where available) a **deadline / enforcement** so updates are not deferred indefinitely. *(On modern macOS, Intune may present Declarative Device Management update-enforcement settings. Availability is version-dependent — use the options Intune shows for your macOS releases; see the Overview version note.)*

1. **Assignments** → **Add groups** → `macOS Baseline – Pilot Devices`. **Review + create** → **Create**.

Step 8 — Create the compliance policy

1. Intune → **Devices** → **macOS** → **Compliance** → **Create policy** → **Platform: macOS** → **Create**.
2. **Name:** `macOS Baseline – Compliance`. **Description:** tag `macOS-Baseline-v1.0`. **Next**.
3. Set the agreed compliance rules, for example: **minimum OS version**, **FileVault** required, **firewall** required, and system integrity where available.

1. **Actions for noncompliance:** mark noncompliant after a short grace period; optionally email the user. **Next**.

1. **Assignments** → **Add groups** → `macOS Baseline – Pilot Devices`. **Review + create** → **Create**.

Step 9 — Add a custom `.mobileconfig` profile (only if needed)

1. For any agreed setting with **no** settings-catalog payload (from Step 1.2), obtain or author the matching `.mobileconfig` file (Requirement 2.8).

1. Intune → **Devices → macOS → Configuration → Create → New Policy → Platform: macOS → Profile type:

Templates → Custom → Create**.

1. **Name:** `macOS Baseline – Custom – Pilot`. **Description:** tag `macOS-Baseline-v1.0`. Provide a deployment channel and **upload** the `.mobileconfig` configuration profile file. **Next**.

1. **Assignments** → **Add groups** → `macOS Baseline – Pilot Devices`. **Review + create** → **Create**.

Step 10 — Apply and validate on the pilot, with pilot gate and rollback note

1. On a pilot Mac, force a sync: **Company Portal** → ■ → **Check status** (or **System Settings → Privacy

& Security → Profiles**, then let it refresh).

1. In Intune, open each pilot policy → **Device status** / **Per setting status** and confirm

Succeeded with no conflicts. Check the compliance policy's **Device status** too.

1. On the Mac (or from an admin workstation) run `MACBASE-Validation.ps1` (per-device applied-state

check) and `MACBASE-Report.ps1` (fleet/pilot posture and drift).

1. **Soak the pilot:** have pilot users run their real apps for several business days. Watch for

anything the baseline changed unexpectedly (login window behaviour, screen-saver/password prompts, a restriction blocking a legitimate workflow, an update deadline interrupting work). Complete the **Testing Guide** (document 5).

1. **Gate:** do not expand until the pilot passes the Testing Guide gate AND no unresolved workflow

breakage remains.

1. **Rollback (if a setting causes a problem):** in Intune, **remove the assignment** of the offending

policy from the pilot group (or set the specific setting to **Not configured**), then sync. Removing the assignment is the fastest safe rollback; the Mac reverts on the next policy cycle. Record the change and the offending setting in your change record.

Step 11 — Expand to production

1. Duplicate each piloted policy (Intune → policy → **Duplicate**) and rename the copies

`macOS Baseline – Config – Production`, `macOS Baseline – Restrictions – Production`, `macOS Baseline – Software Update – Production`, and (if used) `macOS Baseline – Custom – Production`. Keep the tag `macOS-Baseline-v1.0` in each description.

1. Reassign the production copies to `macOS Baseline – Production Devices`. Assign

`macOS Baseline – Compliance` to production too (add the production group to the existing compliance policy).

1. Roll out in **waves** if the fleet is large; re-run `MACBASE-Report.ps1` after each wave to confirm

posture and catch drift. Optionally run `MACBASE-Set-ScopeTargeting.ps1` to add/stage group assignments the repeatable way (defaults to `-WhatIf`).

Order of operations

`Confirm APNs/enrollment → Groups → Baseline the tenant → Config profile → Restrictions → Software Update → Compliance → (Custom .mobileconfig) → Apply + Validate (pilot gate / rollback) → Expand`

5. Testing Guide — macOS Endpoint Management Baseline (Microsoft Intune)

Run after the pilot objects apply (Implementation Step 10), and again after any baseline change. Reuse the **exact** object names from the Implementation Guide.

Test 1 — Baseline configuration profile applied to the pilot

Do: Intune → **Devices** → **macOS** → **Configuration** → `macOS Baseline – Config – Pilot` → **Device status** (and **Per setting status**). **Expected:** Pilot Macs show **Succeeded**; per-setting status shows the baseline settings applied with no conflicts.

Test 2 — Restrictions policy applied

Do: Intune → **Devices** → **macOS** → **Configuration** → `macOS Baseline – Restrictions – Pilot` → **Device status**. **Expected:** Pilot Macs show **Succeeded**; the agreed restrictions (e.g. AirDrop, content caching) are in effect on the Mac.

Test 3 — Software-update policy applied

Do: Intune → **Devices** → **macOS** → **Configuration** → `macOS Baseline – Software Update – Pilot` → **Device status**; on a pilot Mac confirm **System Settings** → **General** → **Software Update** reflects the managed behaviour. **Expected: Succeeded**; automatic download/install and the minimum-OS/deadline settings are present.

Test 4 — Settings present on the device

Do: On a pilot Mac (or from an admin workstation) run `MACBASE-Validation.ps1`. **Expected:** The script reports the Mac's baseline objects as **applied**/succeeded with no drift flagged.

Test 5 — Compliance policy evaluates the device

Do: Intune → **Devices** → **macOS** → **Compliance** → `macOS Baseline – Compliance` → **Device status**; or on the Mac check **Company Portal**. **Expected:** Pilot Macs evaluate **Compliant** against the agreed rules (e.g. minimum OS version, FileVault on, firewall on).

Test 6 — Custom `.mobileconfig` profile applied (if used)

Do: Intune → `macOS Baseline – Custom – Pilot` → **Device status**; on the Mac confirm via **System Settings** → **Privacy & Security** → **Profiles**. **Expected:** The custom profile shows **Succeeded** and the payload is present on the Mac.

Test 7 — Workflow soak (the baseline did not break the business)

Do: Pilot users run their real apps and daily workflows for several business days (login window, screen-saver/password prompts, any restricted feature they legitimately use, the update deadline). **Expected:** No workflow breakage. Any breakage is traced to a specific setting and handled by adjusting it or rolling back (Implementation Step 10.6).

Test 8 — Fleet posture and drift report

Do: Run `MACBASE-Report.ps1` (optionally with `-CsvPath`). **Expected:** A clear per-object breakdown of device states and a drift count; pilot Macs appear succeeded/compliant.

Test 9 — Remediation dry-run (safe by default)

Do: Run `MACBASE-Remediation.ps1 -PolicyNameLike "macOS Baseline"` (no `-WhatIf:$false`). **Expected:** With `-WhatIf` (the default), the script **lists** the drifted Macs and the sync actions it *would* issue — and makes **no** change — proving the remediation path before you ever apply it.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass; no unresolved workflow breakage on the pilot
Production pass	Tests 1–5 and 8 pass for `macOS Baseline – Production Devices` after each rollout wave
Change pass	After any baseline change: Tests 1–7 re-pass on the pilot before promoting to production

6. Troubleshooting Guide — macOS Endpoint Management Baseline (Microsoft Intune)

Field issues and how to resolve them. Reuse the **exact** object names from the Implementation Guide. Start most investigations with `MACBASE-Discovery.ps1` (readiness) and `MACBASE-Report.ps1` (posture).

A. Nothing applies to any Mac

Symptoms: No baseline policy shows a device status; Macs never receive profiles. **Likely causes & fixes:**

- **APNs certificate missing or expired.** Without a valid Apple MDM Push certificate, **no** profile, compliance policy, or update policy reaches any Mac. Intune → **Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate**; confirm present and **not expired** (Requirement 2.3). `MACBASE-Discovery.ps1` reports expiry. Renew with the **same** Apple ID used to create it — a different Apple ID forces re-enrollment of every Mac.
- **Macs not actually enrolled/managed.** Intune → **Devices** → **macOS** → **macOS devices**; confirm the target Macs appear as managed (Requirement 2.4).
- **Policy assigned to an empty or wrong group.** Confirm the pilot Macs are members of `macOS Baseline – Pilot Devices` and that the policy's **Assignments** include that group.

B. A single Mac is not applying the baseline

Symptoms: Most pilot Macs succeed; one does not. **Fixes:**

- **Group membership.** Confirm the Mac is in the assigned group (dynamic-group membership can lag —

give it time or re-evaluate).

- **Force a sync.** On the Mac: **Company Portal** → ■ → **Check status**, or **System Settings → Privacy &

Security → Profiles** and let it refresh. Then run `MACBASE-Validation.ps1 -DeviceName the-mac-name`.

- **Stale/duplicate device record.** A re-imaged Mac can leave an orphaned record. Remove the stale

record so status reflects the live device.

C. Device status shows "Error" or "Conflict"

Symptoms: A baseline policy reports **Error** or **Conflict** on the Per-setting status. **Fixes:**

- **Conflict = two authorities.** The same Apple payload is being set by **two** policies (for example

a legacy "Device restrictions" template and your new settings-catalog restrictions policy, or two catalog policies). Find both and **remove the setting from one**. Run `MACBASE-Discovery.ps1` to list every macOS policy touching the area before you build, to avoid this.

- **Error = payload rejected.** Often a value the current macOS version does not accept, or a payload

requiring supervision the Mac lacks. Check the Per-setting detail, confirm the value against Apple's documentation for that macOS version, and adjust.

D. Restrictions not taking effect

Symptoms: A restriction (e.g. AirDrop, content caching) is set in Intune but not enforced on the Mac. **Fixes:**

- Confirm `macOS Baseline – Restrictions – Pilot` shows **Succeeded** on the device (Testing Test 2).
- Some restriction payloads apply **only to supervised** (ADE-enrolled) Macs. A user-enrolled Mac may

ignore them by design — verify the payload's supervision requirement in Apple's documentation. This is a platform constraint, not a misconfiguration.

- Check for a **conflicting** profile also setting that payload (see C).

E. Software updates not enforcing

Symptoms: `macOS Baseline – Software Update – Pilot` shows Succeeded but Macs do not update, or the deadline never fires. **Fixes:**

- **macOS version dependency.** Update-enforcement behaviour — especially Declarative Device

Management-based enforcement — depends on the macOS major version. Confirm your Macs are on a version that supports the setting you configured (`MACBASE-Discovery.ps1` shows the version spread); on older macOS, some enforcement options simply do not apply.

- **No update available / already current.** Nothing enforces if the Mac is already at or above the target version.
- **Network to Apple update servers.** The Mac needs outbound access to Apple's software-update endpoints; a proxy/firewall block prevents downloads.

F. Compliance policy marks Macs non-compliant unexpectedly

Symptoms: Macs you expected to be compliant evaluate **non-compliant** against `macOS Baseline – Compliance`. **Fixes:**

- **FileVault rule but FileVault not on.** If the policy requires FileVault, a Mac without encryption is correctly non-compliant — deploy encryption (a separate profile/solution) or adjust the rule.
- **Minimum OS version set above the fleet.** Lower the threshold or update the Macs.
- **Grace period.** A freshly-assigned device is non-compliant until it reports in and the grace period elapses — allow a sync cycle, then recheck **Company Portal**.

G. Custom `.mobileconfig` profile shows applied but the setting is wrong

Symptoms: `macOS Baseline – Custom – Pilot` reports **Succeeded**, but the intended behaviour is absent. **Fixes:**

- A custom profile is **opaque** — Intune reports "applied", not whether the payload is correct. Validate the payload keys/values against **Apple's public MDM documentation**; a wrong key applies silently with no effect. Prefer moving the setting to the **settings catalog** if a catalog payload exists (per-setting status makes errors visible).
- Confirm the **deployment channel** (device vs user) matches the payload's requirement.

H. Drift after a Mac was compliant

Symptoms: `MACBASE-Report.ps1` shows Macs that previously succeeded now drifted. **Fixes:**

- A user removed a profile, a conflicting policy was added, or an apply failed. Run `MACBASE-Remediation.ps1 -PolicyNameLike "macOS Baseline"` (preview first, then `-WhatIf:$false`) to re-sync the drifted Macs so the baseline re-applies.
- If a Mac **keeps** drifting, there is a persistent conflict (section C) — resolve the second authority rather than repeatedly re-syncing.

I. Scripts fail to connect or return nothing

Symptoms: A `MACBASE-*` script errors on `Connect-MgGraph`, or returns no objects. **Fixes:**

• **Consent/scopes.** On first run, sign in with an account that can consent to the read scopes (Discovery/Validation/Report) or the read-write scope (`MACBASE-Set-ScopeTargeting.ps1` needs `DeviceManagementConfiguration.ReadWrite.All`). An Intune Administrator or equivalent is typically required.

• **Graph SDK missing.** The scripts install `Microsoft.Graph` for the current user if absent; if the install is blocked, install it manually (`Install-Module Microsoft.Graph -Scope CurrentUser`).

• **Name-prefix mismatch.** The scripts find objects by the `-PolicyNameLike` prefix (`macOS Baseline` by default). If you renamed the policies, pass the matching `-PolicyNameLike`.

• `-WhatIf` **returned no change (expected).** `MACBASE-Remediation.ps1` and `MACBASE-Set-ScopeTargeting.ps1` default to preview — pass `-WhatIf:$false` to apply.

J. Rollback

Symptoms: A baseline setting broke a legitimate workflow on the pilot. **Fix:** In Intune, **remove the assignment** of the offending policy from `macOS Baseline – Pilot Devices` (or set the specific setting to **Not configured**), then sync. Removing the assignment is the fastest safe rollback — the Mac reverts on the next policy cycle. Record the change and the offending setting, then re-pilot the corrected policy.

First moves for any issue

1. `MACBASE-Discovery.ps1` — APNs, enrollment, and conflicting-policy check.
2. `MACBASE-Report.ps1` — where the drift/errors are, by object.
3. `MACBASE-Validation.ps1 -DeviceName <mac>` — deep-dive one Mac.
4. Resolve conflicts (section C) before re-syncing; use `MACBASE-Remediation.ps1` to re-apply.

7. Best Practices — macOS Endpoint Management Baseline (Microsoft Intune)

Field-proven recommendations for building and operating a macOS baseline in Intune.

Naming and structure

• **Name every object, and reuse the names verbatim.** Use the convention from this solution: `macOS Baseline – <Area> – Pilot` / `– Production`, `macOS Baseline – Compliance`, `macOS Baseline – Pilot Devices` / `– Production Devices`. Consistent names make status pages, reports, and the included scripts readable.

• **Put a version/scope tag in every description** (`macOS-Baseline-v1.0`). It is the fastest way to identify which policies belong to this baseline when a tenant has dozens.

- **One area per policy.** Keep the core configuration, restrictions, and software-update settings in **separate** settings-catalog policies. You can then assign, adjust, or roll back one area without disturbing the others.

Prefer the settings catalog

- **Build in the settings catalog first; reach for a custom `.mobileconfig` only when the catalog cannot express the setting.** The catalog is discoverable, reports per-setting status, and is far easier to audit than an opaque uploaded profile.
- When you *must* use a custom `.mobileconfig`, author it from **Apple's public MDM payload documentation**, keep the source file in change control, and document exactly which setting it delivers (Intune only reports "applied", not per-setting detail, for custom profiles).

Avoid conflicting authorities

- **Do not configure the same setting in two places.** Two policies (or a legacy template plus a catalog policy) setting the same payload is the top cause of **Conflict / Error** device status. Run `MACBASE-Discovery.ps1` before you build to find existing macOS policies that already touch your target areas.
- If a setting shows **Conflict**, find both authorities and remove the setting from one — do not just re-push.

Group and assignment strategy

- **Assign to device groups, not user groups,** for a device baseline — it applies at enrollment regardless of who signs in.
- Use a **dynamic device group** (e.g. filtered on the Mac device attribute) for `macOS Baseline – Production Devices` so new Macs inherit the baseline automatically. Keep the pilot group **static and small**.
- Use `MACBASE-Set-ScopeTargeting.ps1` (defaults to `-WhatIf`) to stage and apply group assignments the repeatable way instead of clicking each policy by hand.

Pilot discipline

- **Always pilot.** Even a "safe" baseline can surprise a workflow (a restriction, a login-window change, an update deadline). A small representative pilot catches it on a few Macs.
- **Soak for several business days** with real apps before expanding. Treat the Testing Guide gate as mandatory, not optional.
- Keep a **rollback note** ready: removing a policy's group assignment is the fastest safe revert.

Software updates

• **Make update behaviour deliberate.** Enforce automatic download/install and a **minimum OS version **so Macs do not fall years behind, but set a** realistic deadline** so an enforced restart never interrupts work without warning.

• Software-update enforcement — especially Declarative Device Management-based enforcement — is **macOS-version-dependent.** Confirm the capability against your fleet's macOS versions before relying on it; configure from what Intune presents.

Compliance and Conditional Access

• **Set compliance rules that mean something** (minimum OS, FileVault, firewall). Compliance is what makes the baseline *provable* and what other systems consume.

• If you later gate access on device compliance, that is **Conditional Access** — a separate solution. Get the baseline compliant first; wire access control second.

Ongoing operation

• **Detect drift on a schedule.** Run `MACBASE-Report.ps1` periodically (and after each rollout wave) and remediate with `MACBASE-Remediation.ps1` (dry-run first).

• **Watch the APNs certificate expiry.** It expires annually; a lapsed APNs certificate silently stops **all** Mac management. `MACBASE-Discovery.ps1` surfaces the expiry — renew well ahead.

• **Record every change** (who, what, rollback) and keep the version tag current.

Scope boundaries (don't reinvent other solutions)

• This is the **general baseline**. Enrollment/ADE, formal CIS/STIG hardening, Defender onboarding, and Conditional Access are **separate** solutions — reference them; do not rebuild them inside the baseline.

8. Deliverables Summary — macOS Endpoint Management Baseline (Microsoft Intune)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	The macOS management model in Intune (MDM/APNs, configuration profiles), settings catalog vs custom `.mobileconfig`, restrictions, compliance policies, macOS software-update management, and the compliance/drift model
2. Requirements Guide	Every prerequisite with Why / Verify / Implement-if-missing (incl. APNs, macOS enrollment, groups, baseline decisions), plus the object-naming convention
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: groups, baseline configuration profile, restrictions, software-update policy, compliance policy, optional custom profile, pilot, validate, rollback, expand
5. Testing Guide	Validation procedures with pass/fail gates (including a safe remediation dry-run)
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Naming, settings-catalog-first, conflict avoidance, pilot discipline, update rings, compliance/CA boundaries

PowerShell scripts

Script	What it does
`MACBASE-Discovery.ps1`	Tenant/device readiness for a macOS baseline: APNs presence/expiry, enrolled Macs, macOS version spread, and existing macOS configuration/compliance/update policies to review for conflicts (read-only)
`MACBASE-Validation.ps1`	Per-device applied-state check — reads Intune device config/compliance states for a named Mac and reports whether the baseline objects applied (read-only)
`MACBASE-Report.ps1`	Fleet baseline posture + drift via Microsoft Graph across the baseline configuration policies, custom profiles, and compliance policy; optional per-device CSV (read-only)
`MACBASE-Remediation.ps1`	Safely reapplies/repairs drifted baseline objects by issuing a device sync and confirming group assignment; `-WhatIf` by default
`MACBASE-Set-ScopeTargeting.ps1`	Adds/stages the pilot or production group assignment on the baseline policies the repeatable way; `-WhatIf` by default

Object naming convention (used across docs and scripts)

- Baseline configuration profile: `macOS Baseline – Config – Pilot` / `– Production`
- Restrictions policy: `macOS Baseline – Restrictions – Pilot` / `– Production`
- Software-update policy: `macOS Baseline – Software Update – Pilot` / `– Production`
- Custom `.mobileconfig` profile (if used): `macOS Baseline – Custom – Pilot` / `– Production`
- Compliance policy: `macOS Baseline – Compliance`
- Device groups: `macOS Baseline – Pilot Devices` / `macOS Baseline – Production Devices`
- Version/scope tag in every policy/profile description: `macOS-Baseline-v1.0`

What the buyer receives

- These guides as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process for building and operating the baseline:

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Every Mac on a consistent, compliant Intune baseline — configuration profiles, restrictions, compliance, and managed software updates — provable per device and proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MACBASE-Discovery.ps1

Purpose: Reports whether the tenant and its Macs are ready to receive the macOS baseline: that an Apple MDM Push (APNs) certificate exists and is not near expiry, how many managed Macs are enrolled, the macOS version spread across the fleet, and which existing macOS configuration, compliance, and update policies already exist (so you avoid creating a conflicting second authority). Read-only against Microsoft Graph (Intune). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACBASE-Discovery.ps1 Requires the Microsoft Graph SDK and these read scopes (consented on first sign-in): DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS APNs certificate presence/expiry, count of enrolled Macs, a macOS-version breakdown, and a list of existing macOS configuration/compliance policies to review for conflicts before you build. Pass = APNs present and not expired, Macs enrolled, no surprise conflicting policies. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\MACBASE-Discovery.ps1 Requires the Microsoft Graph SDK and these read scopes (consented on first sign-in): DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS APNs certificate presence/expiry, count of enrolled Macs, a macOS-version breakdown, and a list of existing macOS configuration/compliance policies to review for conflicts before you build. Pass = APNs present and not expired, Macs enrolled, no surprise conflicting policies. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

MACBASE-Remediation.ps1

Purpose: When MACBASE-Validation / MACBASE-Report shows drift, this makes the Intune-managed macOS baseline objects re-apply to the affected Macs: it issues a device sync action (so the configuration policies, custom profiles, and compliance policy re-evaluate) to each drifted Mac. It does NOT hand-edit settings on the Mac — Intune remains the single source of truth. It is safe by default: it previews unless you pass -WhatIf:$false. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\MACBASE-Remediation.ps1 -PolicyNameLike "macOS Baseline" Apply: powershell -ExecutionPolicy Bypass -File .\MACBASE-Remediation.ps1 -PolicyNameLike "macOS Baseline" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.PrivilegedOperations.All (for the sync action). EXPECTED RESULTS With -WhatIf (default): lists the drifted Macs and the sync actions it WOULD issue, no change. With -WhatIf:$false: issues a sync to each drifted Mac so the baseline objects re-apply. Re-run MACBASE-Validation.ps1 after a policy cycle to confirm the drift cleared. .NOTES White-label — the repeatable way. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$PolicyNameLike = 'macOS Baseline')

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\MACBASE-Remediation.ps1 -PolicyNameLike "macOS Baseline" Apply: powershell -ExecutionPolicy Bypass -File .\MACBASE-Remediation.ps1 -PolicyNameLike "macOS Baseline" -WhatIf:$false Requires the Microsoft

Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.PrivilegedOperations.All (for the sync action). EXPECTED RESULTS With -WhatIf (default): lists the drifted Macs and the sync actions it WOULD issue, no change. With -WhatIf:$false: issues a sync to each drifted Mac so the baseline objects re-apply. Re-run MACBASE-Validation.ps1 after a policy cycle to confirm the drift cleared. .NOTES White-label — the repeatable way. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$PolicyNameLike = 'macOS Baseline')

MACBASE-Report.ps1

Purpose: Reports, across the Mac fleet, how each device evaluates against the macOS baseline objects in Intune — by reading the device statuses for the named settings-catalog policies (config, restrictions, software update), any custom configuration profiles, and the baseline compliance policy. Summarizes succeeded/compliant vs drifted (error/conflict/failed/etc.) per object, with an optional per-device CSV. Read-only against Microsoft Graph. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACBASE-Report.ps1 -PolicyNameLike "macOS Baseline" -CsvPath C:\Temp\macbase-posture.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A per-object breakdown of device states and a total drift count; optional per-device CSV. Pass = drift count is zero (every Mac succeeded/compliant on every baseline object). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$PolicyNameLike = 'macOS Baseline', [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\MACBASE-Report.ps1 -PolicyNameLike "macOS Baseline" -CsvPath C:\Temp\macbase-posture.csv Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A per-object breakdown of device states and a total drift count; optional per-device CSV. Pass = drift count is zero (every Mac succeeded/compliant on every baseline object). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$PolicyNameLike = 'macOS Baseline', [string]$CsvPath)

MACBASE-Set-ScopeTargeting.ps1

Purpose: Assigns a target Entra group (your pilot or production Mac group) to the named macOS baseline settings-catalog policies the repeatable way, so you do not click each policy by hand when expanding. It resolves the group by display name, finds the baseline policies by name prefix, and adds an include-group assignment to each. Safe by default: it previews unless you pass -WhatIf:$false. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\MACBASE-Set-ScopeTargeting.ps1 -GroupName "macOS Baseline - Production Devices" Apply: powershell -ExecutionPolicy Bypass -File .\MACBASE-Set-ScopeTargeting.ps1 -GroupName "macOS Baseline - Production Devices" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.ReadWrite.All + Group.Read.All. (Use the exact group display name from your tenant.) EXPECTED RESULTS With -WhatIf (default): lists the baseline policies and the assignment it WOULD add, no change. With -WhatIf:$false: adds the include-group assignment to each baseline policy. Pass = each baseline policy now lists the target group under Assignments in Intune. .NOTES White-label — the repeatable way. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][string]$GroupName, [string]$PolicyNameLike = 'macOS Baseline')

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\MACBASE-Set-ScopeTargeting.ps1 -GroupName "macOS Baseline - Production Devices" Apply: powershell -ExecutionPolicy Bypass -File .\MACBASE-Set-ScopeTargeting.ps1 -GroupName "macOS Baseline - Production Devices" -WhatIf:$false Requires the Microsoft Graph SDK and DeviceManagementConfiguration.ReadWrite.All + Group.Read.All. (Use the exact group display name from your tenant.) EXPECTED RESULTS With -WhatIf (default): lists the baseline policies and the assignment it WOULD add, no change. With -WhatIf:$false: adds the include-group assignment to each baseline policy. Pass = each baseline policy now lists the target group under Assignments in Intune. .NOTES White-label — the repeatable way. Defaults to -WhatIf (no change unless -WhatIf:$false). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][string]$GroupName, [string]$PolicyNameLike = 'macOS Baseline')

MACBASE-Validation.ps1

Purpose: Checks, for one named Mac, whether the macOS baseline objects have applied: it reads the device's configuration-policy states for the named baseline policies (config, restrictions, software update, and any custom profile) and the device's evaluation of the baseline compliance policy, then reports applied/succeeded vs error/conflict/failed. Use it on a pilot Mac after assignment and any time you suspect drift. Read-only against Microsoft Graph. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACBASE-Validation.ps1 -DeviceName your-mac-name Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-object line for the named Mac (SettingsCatalog / CustomProfile / Compliance) with its state. Pass = every baseline object shows succeeded/compliant; fail = any error/conflict/failed. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$DeviceName, [string]$PolicyNameLike = 'macOS Baseline')

How to run: powershell -ExecutionPolicy Bypass -File .\MACBASE-Validation.ps1 -DeviceName your-mac-name Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-object line for the named Mac (SettingsCatalog / CustomProfile / Compliance) with its state. Pass = every baseline object shows succeeded/compliant; fail = any error/conflict/failed. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$DeviceName, [string]$PolicyNameLike = 'macOS Baseline')